

강화학습

Week2 HW

1. Stationary

- a. 코드 분석
- b. 결과 분석

2. Non-stationary

- a. 코드 분석
- b. 결과 분석

3. 전체 결과 분석

Stationary

1. Stationary – 코드 분석

- Bandit.py

```
class Bandit:
    def __init__(self, arms=10):
        self.rates = np.random.rand(arms)

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

```
class Agent:
    def __init__(self, epsilon, action_size=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(action_size)
        self.ns = np.zeros(action_size)

    def update(self, action, reward):
        self.ns[action] += 1
        self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

- Bandit

팔(arm)마다 고정된 승률(rate)을 갖는 Bandit 환경.

- Agent

표본 평균(sample average) 방식으로 가치(Q)를 갱신하는 epsilon-greedy 에이전트.

1. Stationary – 코드 분석

- parameter_epsilon.py

```
runs = 200
steps = 1000
epsilons = [0.1, 0.3, 0.01]

for eps in epsilons:
    all_rates = np.zeros((runs, steps))

    for run in range(runs):
        bandit = Bandit()
        agent = Agent(eps)
        total_reward = 0
        rates = []

        for step in range(steps):
            action = agent.get_action()
            reward = bandit.play(action)
            agent.update(action, reward)
            total_reward += reward
            rates.append(total_reward / (step + 1))

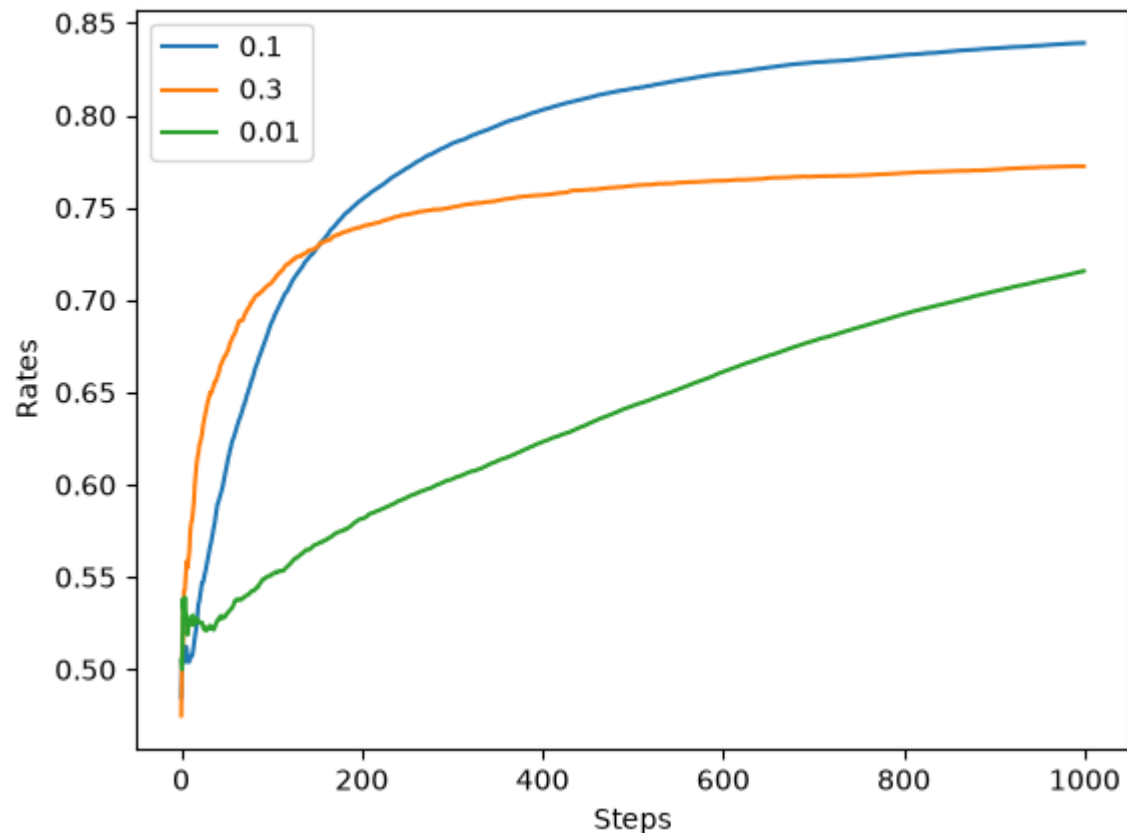
        all_rates[run] = rates

    avg_rates = np.average(all_rates, axis=0)
    plt.plot(avg_rates, label=str(eps))
```

- Parameter_epsilon

정상 환경(Bandit)에서 epsilon 값을 0.1, 0.3, 0.01로 바꿔가며 성능을 비교 및 그래프 출력.

1. Stationary – 결과 분석



- result

최종성능은 $0.1 > 0.3 > 0.01$ 순임.

epsilon = 0.3은 탐험 비율이 높아 초반에 가장 빠르게 좋은 팔을 찾아내나, 학습 종료 후에도 30% 무작위 행동을 지속해 최종 성능은 오히려 낮음(과도한 탐험으로 인한 손실).

epsilon = 0.01은 탐험이 거의 없어 최적 팔 탐색에 가장 오래 걸리고, 1000step 시점까지도 완만한 상승을 지속하며 다른 설정을 따라잡지 못함(탐험 부족으로 학습 지연).

epsilon = 0.1은 탐험-활용 균형이 가장 좋아 세 설정 중 최고 성능.

Non-stationary

2. Non-stationary – 코드 분석

- Non_stationary.py

```
class NonStatBandit:
    def __init__(self, arms=10):
        self.arms = arms
        self.rates = np.random.rand(arms)

    def play(self, arm):
        rate = self.rates[arm]
        self.rates += 0.1 * np.random.randn(self.arms) # 노이즈 추가
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

```
class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions)
        self.alpha = alpha # 고정값  $\alpha$ 

    def update(self, action, reward):
        #  $\alpha$ 로 갱신
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

- NonstatBandit

매 스텝마다 각 팔의 승률이 랜덤하게 흔들리는 비정상 (non-stationary) bandit 환경.

- AlphaAgent

고정된 학습률 α 로 Q값을 갱신하는 epsilon-greedy 에이전트(최근 보상에 더 큰 가중치).

2. Non-stationary – 코드 분석

- parameter_alpha.py

```
runs = 200
steps = 1000
epsilon = 0.1
alphas = [0.2, 0.5, 0.8]

agent_types = ['sample average'] + [f'alpha={a}' for a in alphas]
results = {}

for agent_type in agent_types:
    all_rates = np.zeros((runs, steps)) # (200, 1000)

    for run in range(runs):
        if agent_type == 'sample average':
            agent = Agent(epsilon)
        else:
            alpha = float(agent_type.split('=')[1])
            agent = AlphaAgent(epsilon, alpha)

        bandit = NonStatBandit()
        total_reward = 0
        rates = []

        for step in range(steps):
            action = agent.get_action()
            reward = bandit.play(action)
            agent.update(action, reward)
            total_reward += reward
            rates.append(total_reward / (step + 1))

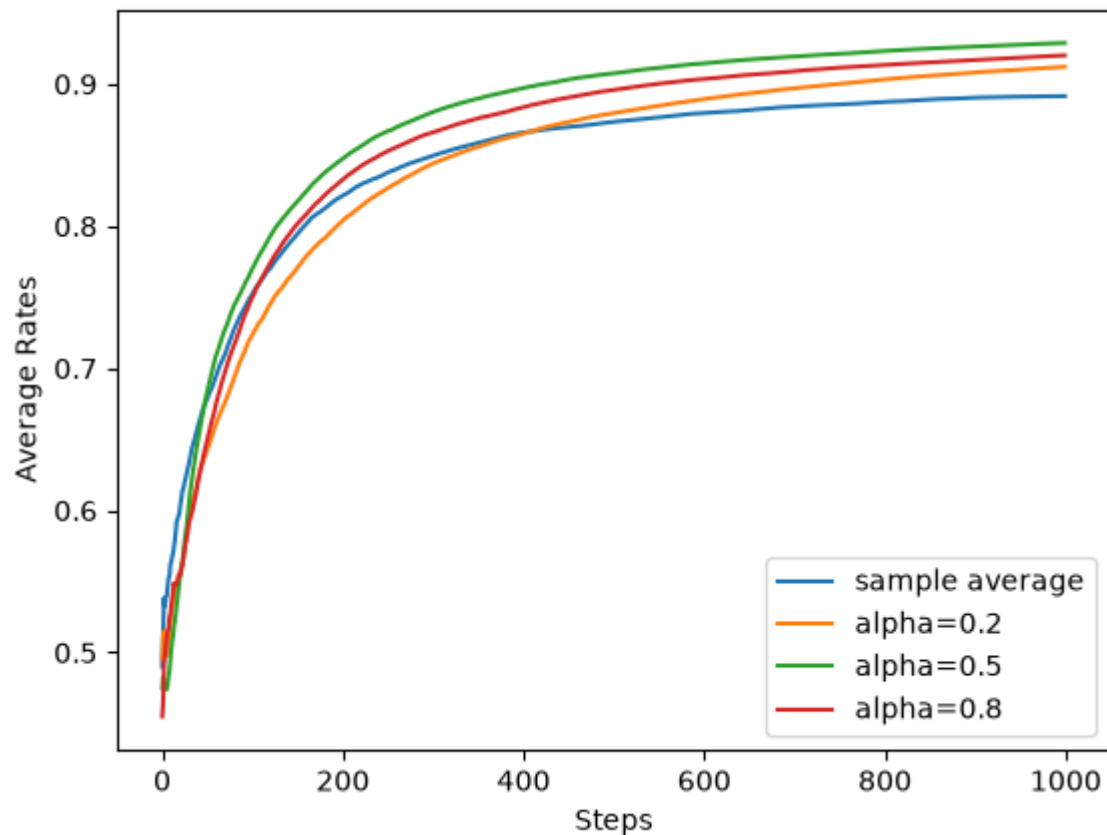
        all_rates[run] = rates

    results[agent_type] = np.average(all_rates, axis=0)
```

- Parameter_alpha

비정상 환경에서 표본 평균 에이전트(Agent)와 다양한 alpha 값(AlphaAgent)을 비교 및 그래프 출력.

2. Non-stationary – 결과 분석



- result

최종성능은 $0.5 > 0.8 > 0.2 > \text{sample average}$ 순임.

sample average는 초반에 비슷한 추세이나, step 진행에 따라 스텝사이즈가 계속 작아져 non-stationary를 추적하지 못하고 가장 낮은 성능에 수렴.

고정 alpha의 AlphaAgent는 최근 보상에 큰 가중치를 두어 환경 변화에 지속 적응, 전반적으로 우수한 성능을 보임.

alpha = 0.2는 적응이 느려 초반 상승은 더딤

alpha = 0.5는 적응-안정성 균형으로 최고 성능

alpha = 0.8은 변동성은 있으나 sample average보다 우수

전체 결과 분석

3. 전체 결과 분석

- epsilon 실험(stationary) :
탐험(exploration) vs 활용(exploitation)의 트레이드 오프, 적당한 탐험($\epsilon=0.1$)이면 충분하며, 지나친 탐험($\epsilon=0.3$)이나 지나친 활용($\epsilon=0.01$) 모두 성능을 떨어뜨림.
- Alpha 실험(Non-stationary) :
적응성(adaptability) vs 안정성(stability)의 트레이드오프, 과거 경험에 동일 가중치를 주는 표본평균 방식보다, 최근 보상에 더 큰 가중치를 주는 고정 alpha 방식이 변화하는 환경을 더 잘 추적함. 단, alpha가 너무 크면(0.8) 노이즈에 민감해져 무조건 클수록 좋은 건 아님.
- 결론 :
bandit 문제에서 좋은 성능을 얻으려면 환경 특성에 맞춰 탐험 비율과 가치 갱신 방식을 적절히 선택해야 함.
즉, 환경이 안정적이면 탐험보다 활용에, 환경이 변한다면 안정성보다 적응성에 무게를 두는 선택이 적절함.